

ACIM-QMM: Efficient Analog Computing-in-Memory Accelerator for QC-MDPC McEliece Cryptosystem

Pingdan Xiao¹, Zhengmiao Wei¹, Sichun Du¹, Wanli Chang^{1*} and Qinghui Hong^{1*}

¹Hunan University, Changsha, Hunan, China

*Email: wanli.chang.rts@gmail.com, hongqinghui@hnu.edu.cn

Abstract—Quasi-cyclic moderate density parity-check McEliece (QMM) cryptosystem is designed to mitigate the security threat posed by quantum computers, and is considered to be a promising candidate for post-quantum cryptography (PQC). However, the growing requirement of data encryption pose severe challenges for QMM implementation in terms of latency and hardware overhead. In this work, we firstly propose ACIM-QMM, an analog computing-in-memory (CIM) accelerator design for QMM cryptosystem. The use of analog circuits and CIM enables the design to efficiently generate key and encrypt ciphertext while breaking the performance bottleneck constrained by digital computing paradigm in PQC. In the experiment, ACIM-QMM can work in low relative error, and it can achieve $31.4\times \sim 288.1\times$ speedup compared with SOTA hardware of QMM cryptosystem. Furthermore, the results indicate that ACIM-QMM can achieve a maximum of $3.12\times$ area efficiency and $20.32\times$ energy efficiency compared to other PQC hardware for 256-bit security.

Index Terms—ReRAM, Circuit design, Analog Computing-in-Memory, QC-MDPC McEliece

I. INTRODUCTION

Different from other post-quantum cryptography (PQC), code-based McEliece cryptography has maintained its security to this day [1], [2]. In the fourth round of NIST PQC standardization process, it was selected as a key encapsulation mechanism finalist [4], [6]. However, code-based cryptosystems inherently grapple with the challenge due to the large key sizes, which impose a significant burden on limited hardware resources and make existing implementations of the McEliece cryptosystem inefficient and impractical [1], [3], [5].

To achieve a compact key while maintaining security, numerous alternatives have been proposed to reduce the size of public key [2]. QC-MDPC McEliece (QMM) cryptosystem stands out as a notable implementation, requiring only 1.2 kilobytes for a 128-bit security level (SL) [5]. Security analyses and research have validated its potential, positioning it as a promising variant within McEliece-type cryptosystems [5]. However, despite its compact nature, further efficient and rapid designs are required to meet the urgent demands of massive data encryption in the context of exponential data growth.

Unlike Lattice-base and Hash-based PQC, code-based QMM cryptosystem requires numerous matrices operation [7], [8]. Therefore, it naturally fits with the crossbar-based computing-in-memory (CIM) circuits for matrices operation, which can avoid the bottleneck of data transmission [9]. Within CIM, different from digital CIM (DCIM), analog CIM (ACIM) circuits can perform matrices operation in one step

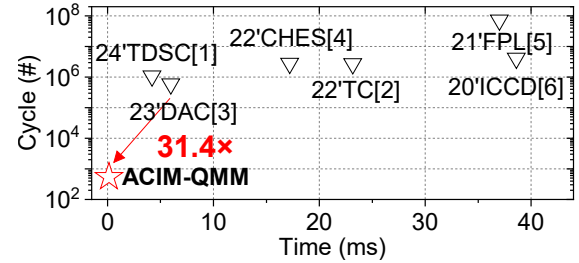


Fig. 1. Comparison with the state-of-the-arts.

due to the computation speed of electromagnetic waves [9]. Additionally, ACIM significantly reduces the use of hardware resources by avoiding signal conversion [10]. However, existing implementation of QMM cryptosystem mainly depends on von-Neumann architecture and digital computing paradigm, the use of which have difficulty in satisfying the aforementioned requirement, and there is no ACIM accelerator design for QMM cryptosystem currently.

Motivated by this, we introduce ACIM-QMM, an analog CIM accelerator leveraging resistive random-access memory (ReRAM) for rapid execution of QMM key generation and encryption. To address the limitation of analog circuits in performing matrix operations in Galois Fields (2) (which is abbreviated as GF(2)), we propose a data-flow deployment and accelerator design. This enables swift GF(2) matrix operations on analog circuits, achieving large-key computations within limited circuits, thereby drastically reducing hardware costs. ACIM-QMM stands out in high-speed, energy-efficient encryption hardware, efficiently tackling any SL task of QMM cryptosystem, outperforming traditional architecture and digital circuits. As illustrated in Fig. 1, ACIM-QMM significantly outpaces other hardware in 128-bit SL evaluations. We believe our design holds great potential for exploring analog circuits to empower PQC. The key contributions of this work are:

- **GF(2) Matrix Deployment:** Our data-flow scheme enables efficient GF(2) matrix operations in analog circuits, handling large-key computations for any SL QMM cryptosystem within limited resources.
- **ACIM-QMM Accelerator:** As the first ReRAM-based ACIM accelerator for QMM cryptosystem, ACIM-QMM significantly accelerates key generation and encryption while lowering hardware costs.
- **Performance Evaluations:** Comprehensive experiments showcase ACIM-QMM's superiority over other hardware, as illustrated in Fig. 1 and Section 4, with the proposed

strategies to mitigate non-ideal analog circuit factors, ensuring robust performance.

II. BACKGROUND AND RELATED WORK

A. Key generation and encryption of QMM cryptosystem

According to the specification of QMM cryptosystem, its procedure of key generation and encryption is shown in Algorithm 1. Notably, the traditional approach to implementing QMM cryptosystems involves crafting gate arrays and digital circuits tailored for each operation. However, as encryption demands grow due to big data, digital circuits and gate arrays face increasing latency and hardware costs. While analog circuits excel in mitigating these issues and executing tasks swiftly and efficiently, they struggle with GF(2) matrix operations. Analog circuits are adept at handling continuous variable problems and can perform matrix calculations in one-step by constructing and mapping mathematical structures. However, unlike digital computing's sequential logical operations, analog circuits cannot establish the necessary mathematical framework for GF(2) matrices, which prevents them from directly executing modulo-2 operations and completing key generation and encryption in QMM cryptosystem within a single matrix operation step. Aiming at the above problem,

Algorithm 1 Key generation and Encryption algorithm in QMM cryptosystem

Require: n, t, r, w, n_0 : Parameters; m : Message; e : Error vector; ($n = n_0 \times r$)

Ensure: H : Private Key; G : Public Key; x : Cipher Text.

- 1: Generate a (n, r, w) -QC-MDPC code $h \in F_2^n$ of weight $w = \sum_{i=0}^1 w_i$ at random.
- 2: $r - 1$ quasi-cyclic shifting h and obtaining parity-check matrix H , where $H = [H_0 | H_1 | \dots | H_{n_0-1}]$.
- 3: Generate public key $G = [H_{n_0-1}^{-1} H_0 | \dots | H_{n_0-1}^{-1} H_{n_0-2} | I_{n \times n}]$ and $G \in F_2^{(n-r) \times n}$.
- 4: Generate $e \in F_2$ of $wt(e) \leq t$ at random.
- 5: Calculate Cipher text $x = mG^T + e$.

we propose a data-flow deployment for GF(2) matrices operation in analog circuits, and ACIM circuits design which can efficiently generate key and ciphertext.

B. Related work

McEliece-type cryptosystems hardware: Current implementations of McEliece cryptography focus on general computing platforms (CPUs, GPUs) and FPGAs, including both classic and QC-MDPC versions. Some relative researches have aimed to reduce ciphertext length for faster encryption [7], [11] and optimize memory usage for better performance [13]. However, the used von-Neumann architecture poses a challenge, limiting the achievement of highly efficient encryption, especially for the requirement of high security levels and large data processing. FPGA implementations have improved speed and power consumption [1], [3], [4], but still face difficulties in maintaining low latency and minimal hardware overhead with increasing data sizes due to the limitations of digital circuits.

CIM architecture for PQC: CIM architecture facilitates in-situ computing within the storage unit, significantly reducing data transmission and hardware overhead, making it ideal for high-speed, efficient hardware-based PQC solutions. Ding et al. proposed an MRAM-based CIM accelerator for LPN-based PQC protocols, achieving speeds 216.8 times faster than CPU and FPGA [14]. Li et al. introduced a CIM-based crossbar design that significantly accelerates polynomial multipliers using the Number Theoretic Transform [15]. These works clearly demonstrate substantial acceleration for PQC. However, they primarily concentrate on digital computing and overlook the benefits of non-volatile devices in analog computing [9]. As computing dimensions and SL escalate, these designs may encounter challenges due to frequent signal conversions and excessive peripheral circuit redundancy.

In summary, the rapid growth of data necessitates a novel hardware design for PQC that can fulfill the increased SL and dimensional data encryption demands.

III. ACIM-QMM DESIGN

In this section, we propose a data-flow deployment and ACIM circuits, which address two issues: (1) enabling analog circuits to directly perform GF(2) matrix operations, including key generation and encryption for QMM cryptosystem; (2) improving efficiency of circuits in latency and performance.

A. GF(2) Matrix Deployment Scheme

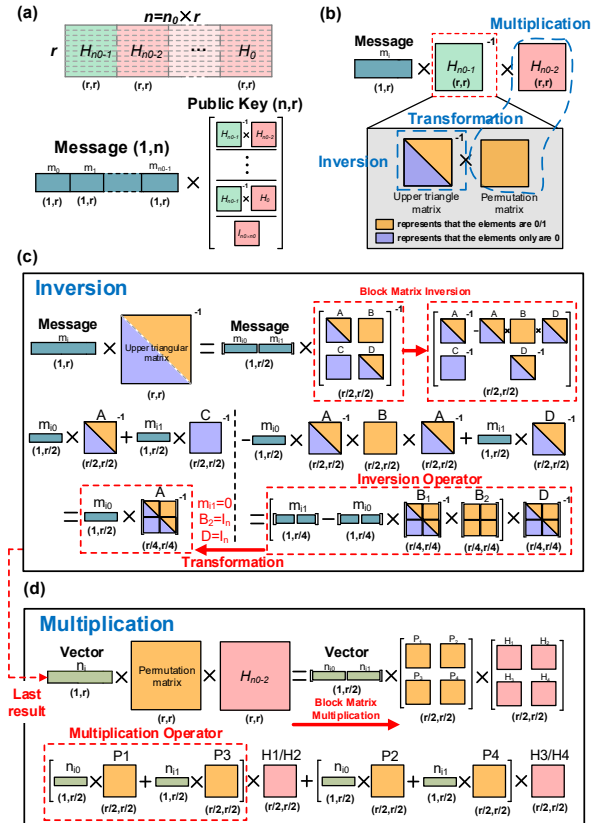


Fig. 2. (a) Parity-check matrix and transformation for H_{n_0-1} ; Example of the matrices inversion (b) and multiplication (c) in key generation adopting the proposed scheme; (d) Procedure of cipher-text encryption.

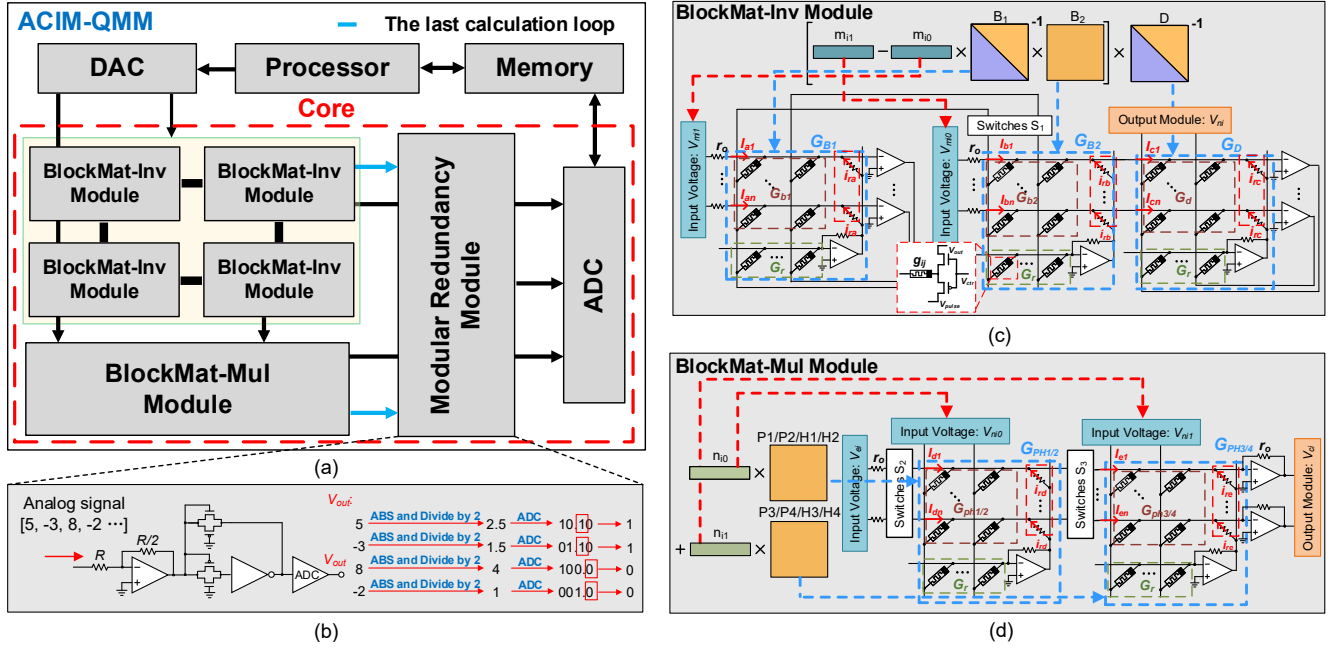


Fig. 3. (a) Overview of ACIM-QMM architecture; (b) Module for modular redundancy operation; (c) BlockMat-Inv circuit for IO in key generation; (d) BlockMat-Mul circuit for MO in key generation and ciphertext encryption.

The generated parity-check matrix H is divided into blocks, with H_{n_0-1} inverted as shown in Fig. 2(a). Algorithm 1 involves multiplying the message m by the public key matrix G built from H . In the process, m can be divided into n_0-1 vector m_i (where $i \in \{0, \dots, n_0 - 1\}$), corresponding to the rows of the block matrices in G . Consequently, multiplying m by G can be broken down into a series of operations: multiplying n_0-2 vectors m_i by matrix $inv(H_{n_0-1})$ and H_i , and directly multiplying m_r by the identity matrix I_{n_0} , as shown in Fig. 2(a).

Given the inherent challenges of matrix inversion in analog circuits over GF(2), we propose a novel approach in the Fig. 2(b). We decompose H_{n_0-1} into an upper triangular matrix and a corresponding permutation matrix, both defined over GF(2). By inverting the upper triangular matrix and then multiplying it by the permutation matrix, we derive the inverse matrix modulo 2. By multiplying each message vector m_i with this inverse matrix and H_i , adding a randomly generated error e , and subsequently combining multiple block matrices, we can efficiently complete the entire key generation and encryption process.

For 80-bit to 256-bit SL encryption tasks, the dimension of H_{n_0-1} ranges from 3079 to 32771. However, limited ReRAM-based crossbars cannot directly handle such large matrix operations. Matrix multiplication can reduce complexity by segmenting matrices into blocks and sequentially performing block matrix multiplication and addition, allowing efficient execution on limited crossbars. However, matrix inversion does not lend itself to straightforward decomposition. To overcome this, we decompose the inversion matrix of the aforementioned upper triangular matrix and its corresponding permutation matrix. The inversion of the upper triangular matrix can be achieved by following these rules:

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & -A^{-1}BD^{-1} \\ 0 & D^{-1} \end{bmatrix} \quad (1)$$

where block matrices A and B are the upper triangle matrices. Based on the rule, we can multiply the message vector m_i multiply with the inversion of upper triangular matrix, which is as demonstrated in Fig. 2(c) and has the following form:

$$m \begin{bmatrix} A & B \\ 0 & D \end{bmatrix}^{-1} = [m_{i0}A^{-1} \quad (m_{i1} - m_{i0}A^{-1}B)D^{-1}] \quad (2)$$

where m_{i0} and m_{i1} are the sub-vector from vector m_i and the dimension relation is shown in Fig. 2(c). In the process of multiplying the inverse matrix of H_{n_0-1} with the message vector m_i the main operation involves $m_{i1} - m_{i0}A^{-1}BD^{-1}$, referred to as the Inversion Operator (IO) in this manuscript. By setting m_{i1} to 0 and matrices B and D to identity matrices, $m_{i0}A^{-1}$ can be calculated using this operator. According to block matrix rules, we can further decompose the operator and continuously reduce the scale of the upper triangular matrix to be compatible with limited-scale crossbars.

The above results following need to multiply permutation matrix P and matrix H_i . This process is illustrated in Fig. 2(c) and it follows:

$$\begin{aligned} nP &= \begin{bmatrix} n_0 & n_1 \end{bmatrix} \begin{bmatrix} P_1 & P_2 \\ P_3 & P_4 \end{bmatrix} \\ &= \begin{bmatrix} n_0P_1 + n_1P_3 & n_0P_2 + n_1P_4 \end{bmatrix} \end{aligned} \quad (3)$$

Through analysis, it is evident that there is a basic operator: $n_0P_1 + n_1P_3$, which can be adapted by replacing the permutation matrix P with matrix H_i to perform operations with H_i , and it is called as Multiplication Operator (MO) in this manuscript. Similarly, the block matrices used can be further decomposed to perform vector-matrices multiplication within them. Consequently, we can implement matrix inversion op-

erations in GF(2) on analog circuits. Based on the proposed operators, we can design domain-specific analog circuits that pave the way for executing key generation and encryption tasks of any SL QMM with limited hardware resources.

B. ACIM-QMM Circuits and Architecture

Using the aforementioned GF(2) matrix deployment scheme, we design analog circuits based on the operator and accelerator architecture shown in Fig. 3. The crossbar employed in our design adopts a 2T1R structure, and the programming method is based on the model described in [10].

The BlockMat-Inv circuit is proposed for mapping with IO, which is primarily composed of three 256×256 crossbar. Specifically, the messages m_{i1} and m_{i0} in IO are mapped to the input voltage source within the module, while the computed matrices B_1 , B_2 and D are mapped to the crossbars, which is illustrated in Fig. 3(c). Since ReRAM cannot directly represent 0/1 elements in GF(2), we propose an equivalent matrix expression circuit comprising sub-crossbars and a common ReRAM row. For instance, G_{B1} is composed of a sub-crossbar G_{b1} and common ReRAM row G_r . Under the same voltage V , the following relation holds:

$$I = VG_{b1} - VG_r \text{vec}(1)^T \quad (4)$$

where I represents the current obtained by applying voltage V into the sub-crossbar G_{b1} in conjunction with the common ReRAM row G_r ; $\text{vec}(1)$ denotes a vector of all ones. Utilizing operational amplifiers (OAs), the current resulting from voltage V applied to G_r can be redirected to flow back into the sub-crossbar G_{b1} . Consequently, we can derive the equivalent matrix $G_{B1} = G_{b1} - G_r \text{vec}(1)^T$. This equivalent matrix G transforms the high and low states of ReRAM into a 0/1 expression. Initially, when voltage V_{mi1} is applied into crossbar G_{B1} , the following principle applies:

$$\begin{cases} V_o = -(V_{mi1}/r_o) \cdot (G_{b1} - G_r)^{-1} \\ V_{mi1}/r_o = I_a + i_{ra} \\ I_a = V_o/G_{b1}, i_{ra} = -V_o/G_r \end{cases} \quad (5)$$

where G_{B1} is the equivalent matrix of the city constructed by G_r and G_{b1} . In there, the use of OAs and negative-feedback principle, the circuit converts the current into the voltage, and it is formally equivalent to taking the reciprocal of conductance G according to the principle of trans-impedance amplifiers (TIA). Based on the equivalent matrix, the sub-circuit achieve matrix inversion of upper triangle matrix B_1 . After obtaining results V_o , under the control of switches, the remainder sub-circuits have the following principles:

$$\begin{cases} V_{ni} = [V_{mi1}/r_o + V_o (G_{b2} - G_r)] \cdot (G_d - G_r)^{-1} \\ V_{mi1}/r_o = I_b + i_{rb} + I_c + i_{rc} \\ I_b = V_o/G_{b2}, I_c = V_{ni}/G_d, i_{rb} = -V_o/G_r, i_{rc} = -V_{ni}/G_r \end{cases} \quad (6)$$

where $G_{b2} - G_r = G_{B2}$ and $G_d - G_r = G_{D2}$. The current flowing into matrix inversion circuit is obtained from voltage V_{mi0} , V_o and crossbar G_{B2} , and then the right sub-circuit converts the input current into voltage. Based on the above all analysis, the BlockMat-Inv circuit has the following relation:

$$V_{ni} = -[V_{mi1}/r_o - S \cdot [V_{mi0}/r_o \cdot G_{B1}^{-1}] \cdot G_{B2}] \cdot G_D^{-1} \quad (7)$$

where V_{ni} is the output, r_o is the resistor which can convert voltage into current, S is the control signal from Switches. The BlockMat-Inv circuit perform the IO operation when $S = 0$, otherwise it can map with the term $m_{i0}A^{-1}$.

The BlockMat-Mul circuit of Fig. 3(d) can achieve the MO operator, and the BlockMat-Mul circuit is composed of 512×512 ReRAM-based crossbar. The aforementioned result voltage V_{ni} is transferred to this circuit, it also divided into two vector n_{i0} and n_{i1} , which are respectively mapped with input voltage source. The block matrices of P and H in Fig. 2 can respectively map with the crossbar $G_{PH1/2}$ and $G_{PH3/4}$. Therefore, the principle is as follow:

$$V_{ci} = -r_o \cdot [S_2 \cdot (V_{ei}/r_o) + I_{ni}] \quad (8)$$

$$\begin{aligned} I_{ni} &= S_3 \cdot (V_{ni0}G_{PH1/2}) + V_{ni1}G_{PH3/4} \\ &= S_3 \cdot (I_d + i_{rd}) + I_e + i_{re} \end{aligned} \quad (9)$$

where S_2 and S_3 respectively represent the Switches. To be specific the above principle, it has the following form:

$$I_{ni} = S_3 \cdot (V_{ni0}G_{ph1/2} - V_{ni0}G_r) + V_{ni1}G_{ph3/4} - V_{ni1}G_r \quad (10)$$

The BlockMat-Mul circuit identically adopts equivalent matrices, it's kernel is to perform MVM operation. It will perform the operation that the multiplication of calculating results and permutation matrix P /block matrix H_i when $S_3 = 1$, the error vector also will be added if $S_2 = 1$ (which achieve map with the Process 9 of Algorithm 1), otherwise it will only perform multiplication of single block matrix.

The modular reduction operation can be the final step after completing all matrix operations, as noted in [16]. Moreover, we also propose a circuit design to perform modular reduction aiming at analog circuits, as shown in Fig. 3(b), which should be used after the other operation. The design consists of switches, OAs, inverters and analog to digital converter (ADC). The module has the following discipline:

$$C(x, y) = \begin{cases} 1, x > y \\ 0, x \leq y \end{cases} \quad (11)$$

$$f(i) = C\left(\left\lfloor \frac{V_c}{2} \right\rfloor, \frac{V_r}{2^n} (2^{i+1} - 2)\right) + C\left(\left\lfloor \frac{V_c}{2} \right\rfloor, \frac{V_r}{2^n} (2^{i+1} - 1)\right) \quad (12)$$

$$V_{final} = \text{sgn} \left(\sum_{i=1}^n f(i) \right) \quad (13)$$

where sgn represents sign function, V_r is the reference voltage of ADC, n is the bits number of ADC. When signal input, it becomes absolute value and divides into 2 by using switches and inverter. After 4-bit ADC conversion, the analog signal converts as digital signal. If 1 exists on the decimal place of the the modular reminder, the result should be 1, otherwise 0.

The above circuits together form the ACIM-QMM architecture, which is shown in Fig. 3(a). In this architecture, we use four BlockMat-Inv Modules for performing IO operation

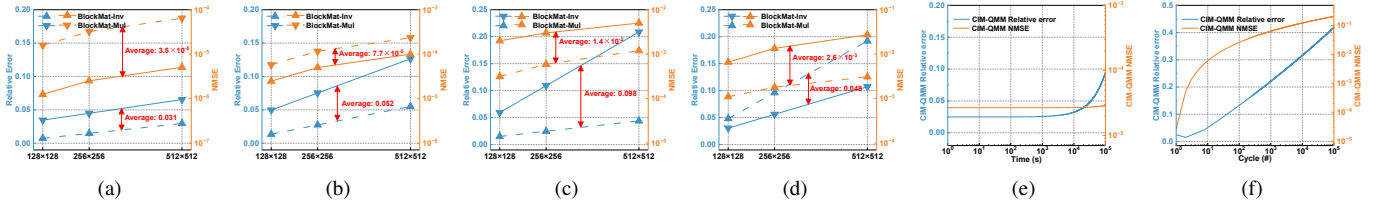


Fig. 4. Relative error and NMSE in the BlockMat-Inv and BlockMat-Mul circuits (a), under wire resistance impact (b), under device deviation impact (c), under noise impact (d), under device retention impact (e) and under device endurance impact (f).

and one BlockMat-Mul Module for performing MO operation. For any SL key generation and encryption task, we can continuously decompose the matrix until it fits the circuit scale of ACIM-QMM based on the above data-flow deployment method, and programming the decomposed block matrices into the crossbar sequentially. The results calculated by the modules of Fig. 3(c) and Fig. 3(d) are transferred into the Modular Redundancy Module, which performs the modulo-2 operation. The obtained results are the generated ciphertext.

IV. RESULTS AND ANALYSIS

ACIM-QMM architecture is simulated for its performance in this section. The circuits are designed and tested in TSMC 180nm and 65nm CMOS library in the Cadence tool. All parameters of the ReRAM cells are set to the values in [17].

A. Evaluation for Calculation

Since ReRAM is easy to be influenced by some non-ideal factors, we test the proposed circuits under various non-ideal factors, as shown in Fig. 4. Fig. 4(a) shows the relative error and normalized mean square root (NMSE) of it, which do not consider non-ideal factor impact, Fig. 4 respectively considers 2.5Ω wire resistance, 2% device deviation and 5% signal noise interference. The above data demonstrates it has good accuracy and minor difference comparing with ideal results.

However, it's worth noting that the relative error from analog circuits may adversely affect ACIM-QMM's performance in key generation and encryption. Accumulated errors can lead to incorrect modular operation results. While small errors in QMM cryptosystem can be tolerated and corrected by the decoding algorithm using the parity-check matrix, excessive errors can severely compromise ciphertext generation. In evaluation, the small NMSE indicates minimal deviation from ideal results. To mitigate error impact, we propose a compensation scheme tailored for QMM cryptosystem. This scheme consists of two parts: adjusting the resistor R in the negative terminal of the OA in Fig. 3(b), and adding a compensating voltage in the Modular Redundancy module. Using Fig. 4(a) as an example, we set R as following:

$$R_{comp} = \frac{R}{1 - RE} \quad (14)$$

where RE is the average value of BlockMat-Inv module's and BlockMat-Mul module's the relative error. Moreover, it can be performed that add complementary voltage in the BlockMat-Mul module, the complementary voltage set as follow:

$$V_{comp} = \frac{\lfloor \lfloor n/\omega \rfloor / r \rfloor}{a} \quad (15)$$

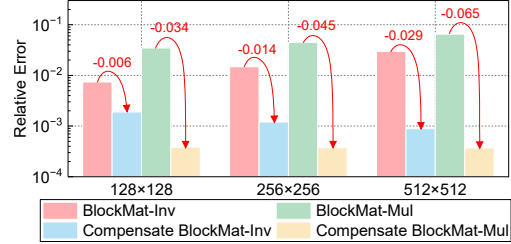


Fig. 5. Compensated accuracy for the proposed modules

where a is the setting parameter; n , r and w are the same as Algorithm 1. Therefore, as shown in Fig.5, the relative error has been reduced to below 10^{-3} and do not impact the results.

In the evaluation of endurance and retention, it can be found that the accelerator under retention has less impact in Fig. 4(e) and it does not influence the final results, but the accelerator under endurance can not be utilized due to large relative error and NMSE in Fig. 4(f). Aiming at the problem, we propose a circuit-level scheme to alleviate the impact, the compensating circuits of Fig. 6(a) is used in the input ports, it can make the input voltage having the same change relation with the conductance of ReRAM under different cycles. Therefore, the input voltage and conductance of ReRAM all obey the identical attenuation curve, which can minimise the relative error and NMSE, as shown in Fig. 6(b).

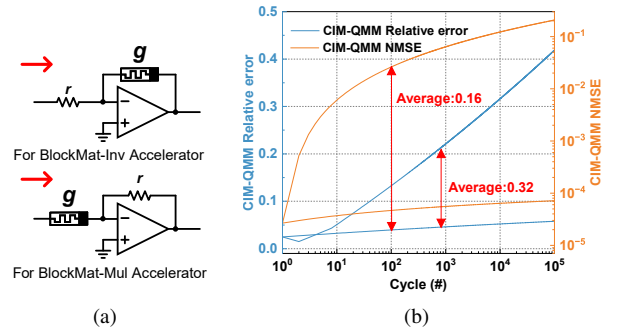


Fig. 6. (a) Compensating circuits for ACIM-QMM; (b) Results.

B. Evaluation for Performance

In this section, we evaluate ACIM-QMM in the area and power consumption, and compare it with other PQC accelerator. Firstly, the area and power consumption are evaluated in Fig. 7, it shows that OAs has the most occupation in power, and DAC and OAs need more area overhead in the design.

Moreover, we compare the performance of ACIM-QMM, CPU and GPU platforms over the standard QMM cryptosystem. The CPU experiments were carried out on an Intel

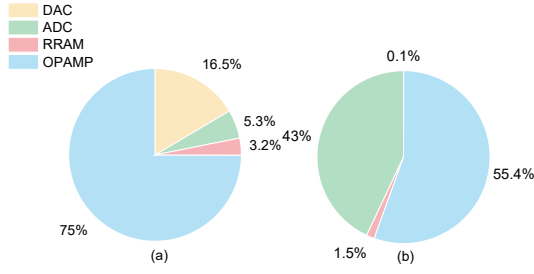


Fig. 7. (a) Power consumption and (b) area consumption of the ACIM-QMM.

TABLE I

PERFORMANCE COMPARISON OF ACIM-QMM WITH OTHER PQC HARDWARE.

Scheme	SL[bit]	Time(<i>ms</i>)		Area (<i>mm</i> ²)	Energy (<i>μJ</i>)
		Gen	Enc./Sig.		
ACIM-QMM	80	0.036		0.42	18.4
	128	0.134			68.6
	256	1.290			660.5
Kyber [18]	128	3.32	4.27	0.67	140.9
	256	7.73	8.96		307.7
Frodo-KEM [19]	128	159	161	0.28	2177.6
	256	994	99		13425.3
Dilithium [20]	128	3.84		1.31	26.6
	256	10.2			82.5

Xeon(R) Platinum 8352V processor. The GPU evaluation is based on the NVIDIA A800-80GB and V100-32GB. All parameters of the tasks are instantiated referencing [7]. Fig. 8 shows that it has great improvement in speed, especially it can achieve maximum $3E6\times$ improvement in 256-bit SL comparing with GPU A800.

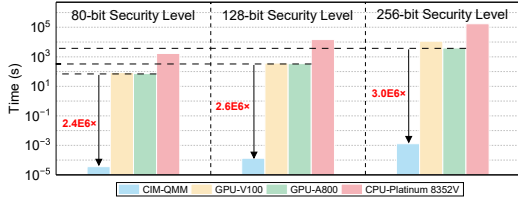


Fig. 8. Comparison with GPU and CPU in executing time.

Table I compares ACIM-QMM with other PQC cryptography hardware, including Kyber, Frodo-KEM, and Dilithium, which offer encryption and signature capabilities. Under a 256-bit SL evaluation, ACIM-QMM achieves a minimum of $7.9\times$ and a maximum of $847.3\times$ improvement in key generation and encryption/signature time. In terms of energy, it offers up to $31.7\times$ and $20.3\times$ improvement for 128-bit and 256-bit SLs, respectively. This significant improvement is attributed to ACIM-QMM's use of ACIM, which performs matrix operations in one-step, drastically lowering power and area overhead. Therefore, we believe that ACIM-QMM can offer a solution for advancing PQC hardware design.

V. CONCLUSION

In summary, we propose an efficient and robust ReRAM-based CIM circuit design for accelerating the QMM cryptosys-

tem, which can rapidly perform key generation and encryption. To address the challenge of analog circuits' inability to perform GF(2) matrix operations, we propose a data-flow deployment method. This method not only resolves the aforementioned issues, but also enables the efficient execution of key generation and encryption for any SL on analog circuits with limited scale. Our experimental results demonstrate that the innovative design can achieve maximum $288.1\times$ speedup compared to SOTA, and maximum $31.7\times$ energy improvement compared to other PQC hardware in 128-bit of security.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (No. 62234008, No. 62371186, No.62471177), Key R&D Program of Hunan Province (No. 2024JK2009), Natural Science Foundation of Hunan Province (No. 2022JJ30160, No.2023JJ30168), Special Funds for the Construction of Innovative Provinces in Hunan Province (No. 2023RC3103, No. 2022SK2007, No. 2023GK2062, No. 2023GK2071), Scientific Research Project of Education Department of Hunan Province (No. 21A0012), Changsha Science and Technology Project (No. KH2202001), Huxiang Young Talents (No. 2023RC3103), Hunan Postgraduate Scientific Research Innovation Project (No. CX20240421).

REFERENCES

- [1] E. Cantó-Navarro et al. AXI Hardware Accelerator for McEliece on FPGA Embedded Systems. *IEEE TDSC*, 2024.
- [2] J. Richter-Brockmann et al. Folding BIKE: Scalable Hardware Implementation for Reconfigurable Devices. *IEEE TC*, 2022.
- [3] Y. Zhu et al. McKeycutter: A high-throughput Key Generator of Classic McEliece on Hardware. in *60th ACM/IEEE DAC*, 2023.
- [4] P. Chen et al. Complete and Improved FPGA Implementation of Classic McEliece. *IACR TCHES*, 2022.
- [5] I. von Maurich et al. Lightweight Code-based Cryptography: QC-MDPC McEliece Encryption on Reconfigurable Devices. In *DATE*, 2014.
- [6] V. Kostalabros et al. HLS-Based HW/SW Co-Design of the Post-Quantum Classic McEliece Cryptosystem In *FPL*, 2021.
- [7] R. Misoczki et al. MDPC-McEliece: New McEliece Variants from Moderate Density Parity-Check Codes. In *IEEE ISIT*, 2013.
- [8] R. Agrawal et al. Quantum-Proof Lightweight McEliece Cryptosystem Co-processor Design. in *38th IEEE ICCD*, 2020.
- [9] S. Wang et al. Smaller, Faster, Lower-power Analog RRAM Matrix Computing Circuits without Performance Compromise. *SCIS*, 2024.
- [10] Q. Hong et al. Memristive Circuit Implementation of Caenorhabditis Elegans Mechanism for Neuromorphic Computing. *IEEE TNNLS*, 2023.
- [11] R. Hooshmand et al. Reducing the Key Length of McEliece Cryptosystem using Polar Codes. in *11th ISCISC*, 2014.
- [12] Y. Li et al. Security Analysis of the Classic McEliece, HQC and BIKE Schemes in Low Memory. *JISA*, 2023.
- [13] B. Colombier et al. Profiled Side-channel Attack on Cryptosystems based on the Binary Syndrome Decoding Problem. *IEEE TIFS*, 2023.
- [14] L. Ding et al. PIMA-LPN: Processing-in-memory Acceleration for Efficient LPN-based Post-Quantum Cryptography. in *60th DAC*, 2023.
- [15] H. Nejatollahi et al. CryptoPIM: In-memory Acceleration for Lattice-based Cryptographic Hardware. in *57th DAC*, 2020.
- [16] William Easttom. *Modern Cryptography: Applied Mathematics for Encryption and Information Security*. Springer, 2022.
- [17] Y. Lu et al. An Electronic Silicon-based Memristor with a High Switching Uniformity. *Nature Electronics*, 2019.
- [18] F. Tim et al. RISQ-V: Tightly Coupled RISQ-V Accelerators for Post-Quantum Cryptography. *IACR TCHES*, 2020.
- [19] B. Utsav et al. Sapphire: A configurable Crypto-processor for Post-quantum Lattice-based Protocols. *IACR TCHES*, 2019.
- [20] P. Karl et al. Post-Quantum Signatures on RISC-V with Hardware Acceleration. *ACM TECS*, 2024.